DELIVERABLE 2

Risk and Technical Debt Inventory

Production-readiness findings, AI-augmented development risks, and actionable backlog integration

Project: flyerrr | Repository: github.com/oanyamele/flyerrr | Prepared: September 10, 2026

Verified result: Six significant debt items and four AI/system risks are documented below. The top three debt priorities are real, assigned GitHub Issues with acceptance criteria and the required tech-debt label.

Objective

Move Flyerrr from a generated prototype toward a production-quality system by identifying structural shortcuts, defining specific remediation, evaluating risks introduced by AI-augmented development, and converting the highest-leverage findings into verifiable backlog work.

VIBE application

Verify	Trace code and data paths; reproduce build, test, and GitHub evidence
Improve	Define trusted boundaries, stronger documentation, and meaningful checks
Build	Implement through owned issues with explicit acceptance criteria
Execute	Require human review and evidence before merge, deployment, or submission

Part 1 - Technical debt audit

TD-01 - Pages are coupled directly to browser storage

Category	Architectural Debt
Severity	P0 - Blocking
Evidence and description	Pages import persistence functions directly and there is no repository or API boundary.
Impact	A backend migration becomes an application-wide cutover; authorization and failure behavior remain inconsistent.
Remediation plan	Define event, attendance, user, bookmark, and verification repositories with HTTP and local adapters; migrate pages incrementally.
Backlog item	Issue #15 - Repository interface

TD-02 - Authentication and authorization are client-side

Category	Architectural Debt
Severity	P0 - Blocking
Evidence and description	Passwords, session roles, and check-in payloads are controlled by browser storage.
Impact	Real student data and organization tools cannot be protected by a client-controlled trust boundary.
Remediation plan	Use managed auth, PostgreSQL Row Level Security, signed credentials, and server-side audit records.
Backlog item	Issues #16 and #18 - Server auth and signed tokens

TD-03 - Tests provide no behavioral confidence

Category	Test Debt
Severity	P1 - High
Evidence and description	The only unit test asserts true equals true; roughly 7,800 source lines lack meaningful coverage.
Impact	A green test command does not detect regressions in data, auth, check-in, or reporting.
Remediation plan	Test repositories and security paths first; enforce at least 60 percent src/data coverage in CI.
Backlog item	Issue #23 - Data-layer tests

Part 1 - Technical debt audit (continued)

TD-04 - End-to-end verification is broken

Category	Test Debt
Severity	P1 - High
Evidence and description	The generated Playwright setup referenced an unavailable package and the core cross-session flow is untested.
Impact	A single-browser demo can pass while the real student-to-organizer workflow remains impossible.
Remediation plan	Restore standard Playwright and verify login, check-in, attendance, and points in separate sessions.
Backlog item	Issue #24 - Cross-session E2E

TD-05 - Setup and architecture knowledge are undocumented

Category	Documentation Debt
Severity	P1 - High
Evidence and description	README.md is a TODO despite Node, Bun, architecture, environment, and workflow requirements.
Impact	Teammates and agents must guess setup and system boundaries, increasing inconsistent changes.
Remediation plan	Document prerequisites, commands, architecture, configuration, and the issue-to-review workflow.
Backlog item	Issue #8 - README and Node requirements

TD-06 - Dependency resolution is nondeterministic

Category	Architectural Debt
Severity	P1 - High
Evidence and description	Three competing lockfiles coexist on main.
Impact	Different package managers can install different dependency graphs.
Remediation plan	Keep bun.lock only, document Bun, and use a frozen Bun install in CI; implementation is In Review.
Backlog item	Issue #9 - Package-manager standardization

Part 2 - AI and system risk assessment

Flyerrr is not currently an AI product at runtime. These risks concern the AI-augmented workflow used to generate, plan, and modify the application.

AR-01 - Plausible but incorrect generated code

Required area	Reliability / Hallucination
Likelihood / impact	High / High
Risk	An agent can invent an API, misunderstand persistence, or implement only a visual success state.
Controls	Require testable issue criteria, reproduce evidence locally, run CI, and trace the full data flow in human review.
Residual risk	Medium until the backend and cross-session E2E exist.

AR-02 - Secrets or personal data exposed to tools

Required area	Security and Ethics
Likelihood / impact	Medium / Critical
Risk	Environment secrets or student records may be copied into prompts, logs, code, or commits.
Controls	Use least privilege, synthetic data, secret scanning, redacted logs, and approval-gated production access.
Residual risk	Medium after server auth and secret scanning; unacceptable before then.

AR-03 - Untrusted text redirects agent behavior

Required area	Security and Ethics
Likelihood / impact	Medium / High
Risk	Repository, issue, or web content can contain instructions that cause unrelated or unsafe actions.
Controls	Treat content as data, scope tools to the task, inspect diffs, and keep destructive or production actions approval-gated.
Residual risk	Low to medium with enforced review and scoped credentials.

AR-04 - Platform and external-service churn

Required area	Dependency Risk
Likelihood / impact	Medium / High
Risk	Lovable conventions, generated packages, provider APIs, or browser features can change or disappear.
Controls	Pin dependencies, isolate providers, record ADRs, test upgrades, and maintain replacement paths.
Residual risk	Medium; interfaces and ADRs reduce but do not eliminate vendor risk.

Part 3 - Backlog integration

The three highest-leverage debt findings are real GitHub Issues on the Flyerrr project board. Their acceptance criteria were refined with AI assistance and checked against repository evidence. Each issue is assigned, estimated, and labeled tech-debt.

1	No repository/API boundary	#15	Backlog	Interfaces, adapters, no direct page imports, unit tests
2	No meaningful data-layer tests	#23	Backlog	Security/data paths and at least 60 percent coverage
3	Missing setup and architecture entry point	#8	Backlog	Node, Bun, scripts, architecture, fresh-clone test

Real GitHub Issues	29
Project status	25 Backlog, 3 In Review, 1 Done
Issues labeled tech-debt	13
Detailed feature backlog	88 rows in docs/backlog.csv
Remaining governance gap	Reconcile CSV rows to live issues; require issue links in PRs

Human-in-the-loop operating rules

1	A named human owns each issue and approves architecture or security decisions.
2	Agent-produced claims are reproduced with local commands or source inspection.
3	Rendered success is not completion; tests must verify state change and failure paths.
4	Secrets, production data, merges, deployments, and submissions remain approval-gated.
5	Every implementation uses an issue-specific branch and review before merge.

Conclusion

Flyerrr's most serious debt is the absence of trusted persistence, authorization, and verification boundaries beneath an otherwise convincing interface. The existing project board now makes the findings actionable. Production readiness should be measured by behavior across independent sessions, not by whether a screen renders.

> **Release test: a student checks in from one session, an authorized organizer validates from another, exactly one attendance row is written, and points and reports update from the same server record.**

Evidence references

docs/PROJECT_RESET_REPORT.md	Full 19-item risk register, impact, fixes, estimates, board audit
docs/architecture.md	Current and target architecture plus broken check-in data flow
docs/backlog.csv	Owners, priorities, dependencies, estimates, and acceptance criteria
Issues #8, #15, and #23	Top-three debt items with tech-debt labels and acceptance checklists
Issues #5-#33	Live issue-backed project board